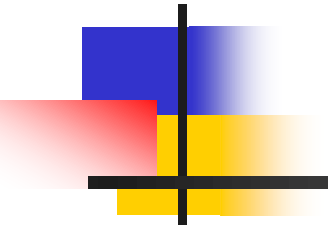
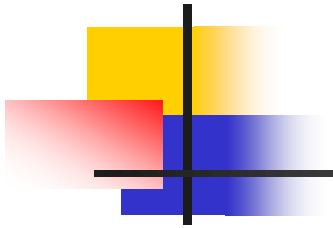
第5章　Verilog HDL程序设计



5.7 Verilog HDL仿真

- 仿真是对一个模块的功能和性能进行评估和检测的方法；
- 仿真一般分为功能仿真和时序仿真。功能仿真检测的是逻辑电路的功能，忽略电路固有的延时特征，可以对系统进行快速评估；
- 时序仿真结合电路固有的延时特性来检测电路的功能，更符合电路的实际情况，但仿真难度大，耗时长。



- Verilog语言中有一部分语句和定义是只针对仿真的，不能综合成硬件电路。
- 这类语句有预编译指令、系统任务和系统函数等；
- 仿真程序也叫做仿真测试平台（Test Bench）

5.7.1预编译指令

- 预编译指令是在模块编译之前，通知编译器进行一些“预处理”，处理的结果再和源代码一起进行编译。
- 预编译指令由反引号（`）开头，其作用范围是从定义开始到文件结束，或遇到新的预编译指令为止。

主要的预编译指令

`define, `undef：文本替代和取消

例如：

```
`define BUS_SIZE 16 //用BUS_SIZE替代16
……
reg [`BUS_SIZE:1] Bus;
……
`undef BUS_SIZE   //此指令之后，BUS_SIZE不再有效
```

主要的预编译指令

`ifdef, `else, `endif：条件编译

例如：

```
`ifdef ADD_BUS //如果定义了宏名ADD_BUS
  pameter BUS_SIZE = 16; //则BUS_SIZE为16
`else
  pameter BUS_SIZE = 0; //则BUS_SIZE为0
`endif
```

主要的预编译指令

`include: 文件包含指令

例如：

```
`include “source_code.v”
`include “D:/decode/primitive.v”
```

//在编译时，该指令会被后面的文件的内容替代。

5.7.2系统任务和系统函数

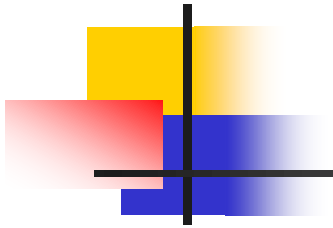
- Verilog语言中提供了用于仿真和测试的内置式系统任务和系统函数
- 系统任务和系统函数都以“$”符号开头；
- 常用的系统任务和系统函数包括：
  - 显示任务
  - 文件输入/输出任务
  - 仿真结束/暂停任务
  - 仿真时间函数

5.7.2.1显示任务

以显示任务$display为例：

- 其作用是根据显示格式要求显示字符串、数值等内容；
- 一般语法格式为：

$display（“带格式的字符串”，参数1，参数2，…）；



例如：

```
$display ("TEST IS FAILURE."); //字符串显示
$display ("A is %d, B is %h", A, B); //数值显示，设A=20，
B=115
```

运行以后显示的结果分别是：

```
TEST IS FAILURE.
A is 20, B is 73
```

在字符串表达式中，有些字符前面的“\”和“%”是转义符，例如：

“\t”表示Tab

“\n”表示换行

“%b”表示二进制格式显示

“%d”表示十进制格式显示

例如：

```
$display (“1\t%d\n2\t%d\\”, A, B); //数值显示，设A=20，B=115
```

运行以后显示的结果是：

```
1       20
2       115\
```

5.7.2.2文件输入/输出任务

以$fopen 、$fclose为例：

系统函数**$fopen**用于打开一个文件，其返回值是文件句柄，即关于文件的一个整数或指针。**$fclose**用于关闭文件。例如：

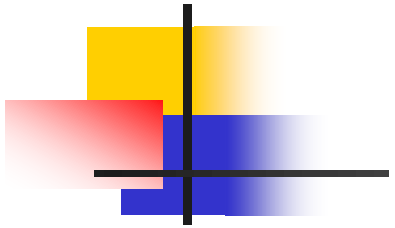
```
integer file_pt;
initial  begin
   file_pt = $fopen ("div.tq");
   ……
   $fdisplay (file_pt,"The simulation time is %t", $time);
   ……
   $fclose (file_pt);
end
```

5.7.2.2文件输入/输出任务

$readmemb**和**$readmemh是对文件进行读取的两个系统任务，它们可以将数据从文件中读出来，并加载到存储器中去。其语句格式为：

$readmemb（“文件名”，存储器名，起始地址，终止地址）；
$readmemh（“文件名”，存储器名，起始地址，终止地址）；

“文件名”和“存储器名”必须有；
“起始地址”和“终止地址”是可选项，用十进制表示。



例如：

```
reg [0:3] MEMA[0:31];
……
initial
  $readmemb ("example.dat", MEMA);//起始单元是存储器定义范围左边的值;
```

若写成：

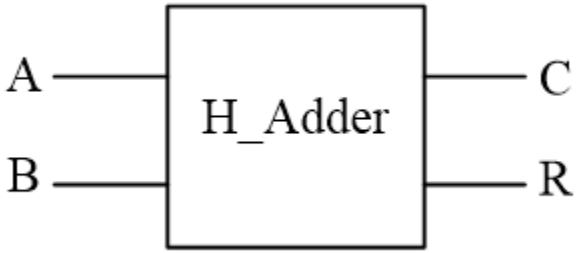
```
  $readmemb ("example.dat", MEMA, 16, 31);
  //数据装载到存储器的起始地址是16，终止地址是31。
```

如果数据文件的数据多于存储器的单元数，则数据存入到该存储器的终止地址为止。反之，未赋值的存储器单元内的值默认为x。

5.7.3 Verilog测试平台

- Verilog测试平台（Test Bench）是指用来测试一个Verilog实体的程序；
- 用Verilog的基本语法要素来编写；
- 测试平台可以产生激励信号，并通过模块例化语句把激励信号传递给被测试的Verilog实体；
- 测试平台可以存储或观察被测试的Verilog实体的输出信号；
- 通过对比，可以判断被测试的Verilog实体的内部结构是否正确。

半加器程序及其测试平台




```
module  H_Adder (A, B, R, C);
    input  A, B;
    output  R, C;

    assign  R = A ^ B;
    assign  C = A & B;
endmodule
```

```
   A
 + B
-----
 C  R
```

半加器测试平台（仿真程序）

```
`timescale 10ns/1ns //仿真时间标度语句必须存在
module H_Adder_tb; //仿真测试平台没有端口
reg a, b;          //测试输入信号，定义为reg型
wire r, c;         //测试输出信号，定义为wire型
initial begin
  a = 0; b = 0;
  #10 begin a = 0; b = 1; end
  #10 begin a = 1; b = 0; end
  #10 begin a = 1; b = 1; end
  #10 begin a = 0; b = 0; end
  #10 $stop;
end
H_Adder U1 (.A(a), .B(b), .R(r), .C(c));
endmodule
```

测试平台说明

- 在写测试平台时，首先给出仿真时间标度语句

```
`timescale;
```

- 测试平台本身也是一个模块，因此要有模块名，如H_Adder_tb，但后面没有端口名列表；
- H_Adder_tb内部设置的信号要跟H_Adder的端口对应。例如，H_Adder_tb内部信号分别为a、b、r、c，与H_Adder的A、B、R、C端口对应。

测试平台说明

- 对应于输入端口的a、b要定义为reg型，对应于输出端口的r、c要定义为wire型；
- H_Adder_tb中采用initial语句对输入端加入激励信号。可以在一个initial语句对a、b加激励信号，也可以用两个initial语句分别对a、b加激励信号；
- 采用模块例化语句将被测试的半加器作为元件放入到测试平台中；
- 激励信号应尽量全面地覆盖输入信号的各种组合情况。

半加器测试平台（另一写法）

```
`timescale 10ns/1ns
module H_Adder_tb;
reg A, B; //内部信号与被测试模块一致
wire R, C;
H_Adder U1 (.A(A),.B(B),.R(R),.C(C));

initial begin
   #0 a = 0;
   #20 a = 1;
   #20 a = 0;
   #10 $stop;
end
```

```
initial begin
   #0 b = 0;
   #10 b = 1;
   #10 b = 0;
   #10 b = 1;
   #10 b = 0;
   #10 $stop;
end
endmodule
```

激励信号编写方式

（1）周期信号

周期信号，如时钟信号，可以采用initial语句和always语句结合的方式产生，例如：

```
initial clk = 0;
always #1 clk = ~clk;
```

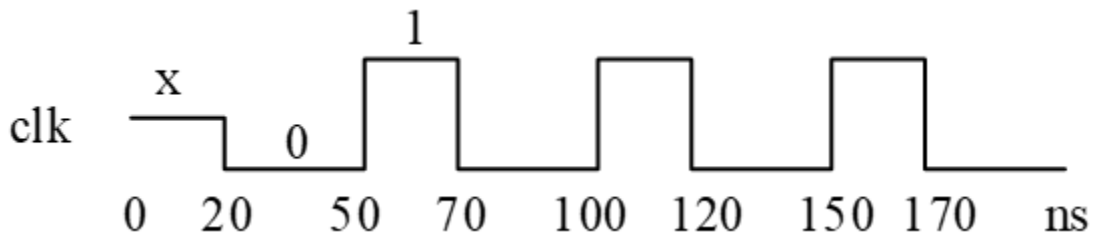
另外，也可以利用forever语句产生周期性激励信号，例如：

```
initial begin
  clk = 0;
forever #1 clk = ~clk;
end
```

激励信号编写方式

占空比非1：1的周期信号，可采用always语句产生，例如：

```
always begin
   #2 clk = 0;
   #3 clk = 1;
end
```



激励信号编写方式

（2）非周期信号

非周期信号的数据量较小时，可以采用initial语句穷举，例如：

```
initial begin
#0 begin a = 0; b = 0; end
#10 begin a = 0; b = 1; end
#10 begin a = 1; b = 0; end
#10 begin a = 1; b = 1; end
#10 begin a = 0; b = 0; end
#10 $stop;
end
```

激励信号编写方式

（2）非周期信号

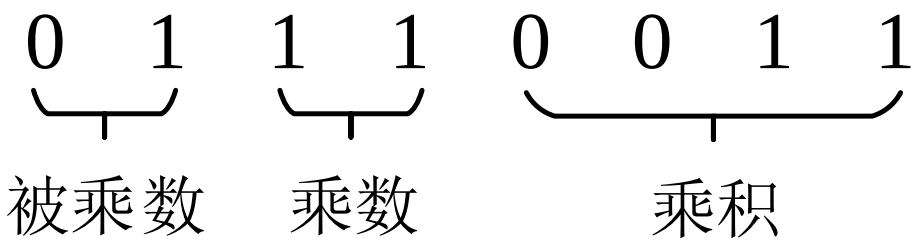
非周期信号的数据量较多时，可以将激励信号存放在一个文件中，调用系统函数读入数据。例如对两位乘法器编写测试平台程序。

例5-36

```
module MUL2_2(M, N, R);
  input [1:0] M, N;
  output [3:0] R;
  ……
end
```

乘法器测试平台

首先把输入激励信号和预期运算结果放在一个数据文件test.dat中，每个数据是8位，格式为：



乘法器测试平台

```
`timescale 1ns/100ps
module MUL2_tb;
reg [1:0] m, n; //测试输入信号，定义为reg型
wire [3:0] r;   //测试输出信号，定义为wire型
reg [1:8] mymem [1:16]; //定义一个8位存储器，
                        //16个单元
reg [3:0] r_EX;
integer J;
```

```
MUL2_2 U1(m, n, r); //模块例化语句，位置关联

initial begin
 $readmemb ("test.dat", mymem);
 for (J=1; J<=16; J=J+1)  begin
   {m, n, r_EX} = mymem[J];
   #5; //延迟5个时间单位，等待电路稳定
   if (r != = r_EX)
     $display ("Mismatch on vector %b", mymem[J]);
   else
     $display ("No mismatch on vector %b", mymem[J]);
 end
end
endmodule
```

仿真数据写入文件

```
integer file_dt;
initial  begin
file_dt = $fopen ("simdata.dat"); //打开一
                                  //个数据文件
……
$fwrite (file_dt, "%b/n", dataout); //将仿
                                    //真数据写入文件
……
$fclose (file_dt);
end
```